

Draft: Improvements on Community Membership Hiding via a Custom Reinforcement Learning Architecture

Abstract

We address the problem of Community Membership Hiding (CMH): given a graph and a community detection algorithm, strategically rewire a target node’s edges so it is no longer grouped into its original community. Building on the Deep Reinforcement Learning (DRL) baseline of (bernini-2024), we identify two fundamental architectural limitations—target-unawareness and static node embeddings—and propose successive improvements culminating in a **Dual-Agent GATv2 Actor-Critic** that specializes in edge addition and deletion separately. We further introduce a *Policy-Guided Subgraphing* heuristic that makes the architecture tractable on large graphs. Empirical evaluation on five benchmark datasets shows consistent and substantial gains in F1 score over all baselines, including the gradient-based methods of (silvestri2025righthidemaskingcommunity).

1 Introduction

The modern science of networks has brought significant advances to our understanding of complex systems. One of the most relevant features of graphs representing real-world structures—ranging from social media platforms to biological networks—is their community structure: tightly connected groups of nodes identified by highly effective algorithms (Fortunato-2010). However, in some contexts this functionality inherently risks exposing individuals to privacy breaches by inadvertently revealing sensitive affiliations, such as political leanings or religious beliefs, based solely on their structural position.

To counter this, the concept of Community Membership Hiding (CMH) has been proposed (bernini-2024). The objective of CMH is to strategically alter the structural properties of a network so that a specific target node is no longer grouped into its original community by a detection algorithm, while maintaining overall graph integrity and treating the algorithm as a black box.

Recent work has formulated this evasion task as a Markov Decision Process (MDP) solved via Deep Reinforcement Learning (DRL) (bernini-2024). While that work demonstrated the viability of the approach, it exhibits notable performance degradation on larger graphs and has fundamental architectural limitations that hinder optimal performance. This paper develops directly from (bernini-2024), dissects its shortcomings, and proposes a novel RL architecture that leverages modern Graph Neural Networks (GNNs). We compare against three strong baselines: the original agent of (bernini-2024), and the gradient-based methods ∇ -CMH and ∇ -CMH (projected) from (silvestri2025righthidemaskingcommunity).

2 Community Evasion

2.1 Problem Definition

The following definitions are borrowed from (bernini-2024). Let $\mathcal{G} = (V, E)$ be a graph where V is the set of nodes and E is the set of edges. Given a community detection algorithm $f(\cdot)$ and a target node

$u \in V$, f assigns u to an original community C_i .

The goal of CMH is to find a modified graph $\mathcal{G}' = (V, E')$ such that upon re-evaluating $f(\mathcal{G}')$ the new community assigned to u , denoted C'_i , is sufficiently dissimilar to the original. Formally, u is considered hidden when:

$$\text{sim}(C_i - \{u\}, C'_i - \{u\}) \leq \tau \quad (1)$$

where $\tau \in [0, 1]$ is a predefined hiding threshold, and $\text{sim}(\cdot, \cdot)$ is a similarity function (we use the Sørensen-Dice coefficient, following the baseline).

To aid hiding, the action space is restricted: the agent may only remove edges from u to nodes $v \in C_i$, and add edges from u to nodes $v \notin C_i$. The maximum number of modifications (edge budget) is:

$$B = \max\left(1, \left\lfloor \frac{|E|}{|V|} \cdot \beta \right\rfloor\right)$$

Beyond hiding success, a desired property is to maximize Normalized Mutual Information (NMI)—a measure of similarity between the original and modified community structures—thereby minimizing structural distortion. We report the **F1 score**, defined as the harmonic mean of the Success Rate (SR) and NMI, as the main evaluation metric.

2.2 Original Agent Architecture

The baseline of (bernini-2024) uses a Graph Convolutional Network (GCN) (kipf2017semisupervisedclassificationgraphconvolutional) to process graph state. Initial node features are generated with *node2vec* (grover2016node2vecscalablefeaturelearning) on the unaltered graph. The agent is constrained to act on the target node via an external environment wrapper that masks invalid actions.

2.3 Architectural Limitations of the Baseline

Two critical bottlenecks limit the baseline’s scalability and effectiveness:

Target-unawareness. The GCN receives the entire graph as input but lacks explicit knowledge of the target node

u . Because standard GNNs are permutation-invariant, the policy $\pi(a \mid s)$ produces identical outputs for any target node given the same graph state. The agent therefore learns a generic “shuffle communities” strategy rather than a node-specific one.

Static embeddings. *Node2vec* embeddings are computed once on $\mathcal{G}$ and never updated. As the agent rewires the graph, these embeddings diverge from the current topology, creating an information lag that degrades learning efficiency.

3 Proposed Architectures

3.1 GATv2 Actor-Critic v1

TODO: agree on a name for this version.

Our first proposed architecture introduces an **Actor-Critic** model with a **shared graph encoder** based on Graph Attention Networks v2 (GATv2) ([brody2022attentivegraphattentionnetworks](#)). Instead of static node2vec embeddings, we define a *target-aware* feature space that is partially recomputed after each rewiring step.

Static features (computed once on the original graph):

- *Is Target* — identifies the node being hidden.
- *In Target Community* — marks the original deception boundary.
- *Is Original Neighbor* — preserves the initial neighborhood context.

Dynamic features (updated after every rewiring action):

- *Normalized degree* — tracks local density changes.
- *Current neighbor status* — binary edge indicator w.r.t. the target.
- *Common neighbors* — structural overlap with the target.
- *Jaccard similarity* to the target.

At the core, a **shared GNN backbone** of stacked GATv2 layers encodes node features and edge attributes into node embeddings $H \in \mathbb{R}^{N \times h}$. These embeddings are consumed by both heads:

- **Actor head.** Computes per-node logits conditioned on the target embedding h_t via a scoring function $\ell_i = f(W_1 h_i + W_2 h_t)$, where h_i is the candidate node embedding. The target node’s own logit is masked to $-\infty$.
- **Critic head.** Combines an attention-pooled global graph representation $h_G = \sum_i \alpha_i h_i$ (with $\alpha_i = \text{softmax}(g(h_i))$) with the target embedding to produce a scalar state-value estimate $V(\mathcal{G}, u)$.

The reward signal follows the CMH objective: a positive terminal reward on successful hiding, a negative terminal reward on budget exhaustion, and step-level costs that differ for edge addition vs. deletion to encourage efficient rewiring.

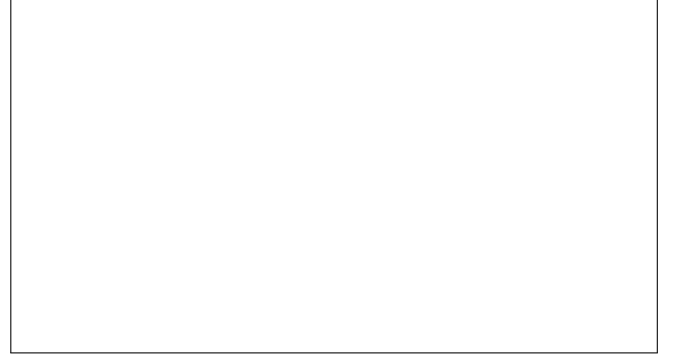


Figure 1: [Placeholder] GATv2 Actor-Critic v1 architecture diagram.

3.2 GATv2 Actor-Critic v2 (Dual-Agent)

TODO: agree on a name for this version.

To further improve performance—particularly NMI preservation and scalability—we move from a single monolithic agent to a **Dual-Agent Architecture** that decomposes the rewiring task into two specialized sub-problems.

Agent Specialization

- **Adder Agent.** Trained exclusively to identify and create optimal inter-community edges from u to nodes outside C_i .
- **Deleter Agent.** Trained exclusively to identify and remove critical intra-community edges from u to nodes inside C_i .

Both agents share the same GATv2-based backbone design (same feature space, same encoder architecture), but maintain *independent* actor and critic heads, each trained on their respective action subspace.

Action-Selection Mechanism

At each timestep, both agents are queried. The agent selected to act is the one whose Critic reports the higher expected state value:

$$\text{act} = \arg \max_{a \in \{\text{add}, \text{del}\}} V_a(\mathcal{G}, u)$$

This value-based selection allows the architecture to adaptively switch between addition and deletion depending on which is more beneficial given the current graph state, without requiring a hand-crafted switching rule.

Scaling via Policy-Guided Subgraphing

A key bottleneck on large graphs is the cost of recomputing dynamic features (Common Neighbors, Jaccard Similarity) for all nodes after every action, which grows with $|V|$ and $|E|$. We address this with a **Policy-Guided Subgraphing** heuristic:

1. **Initial pass.** Perform a single forward pass of both Actor networks on the full graph to obtain raw logits.

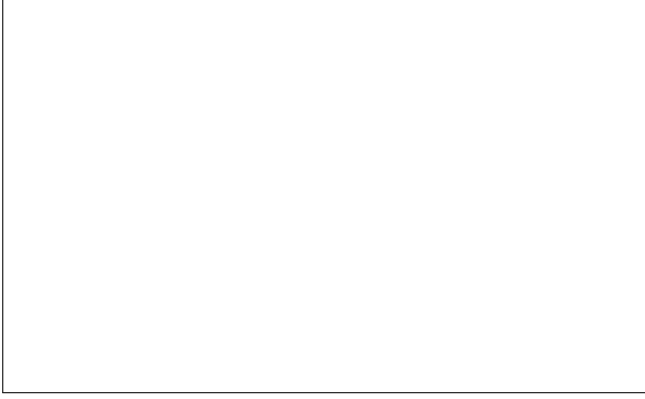


Figure 2: [Placeholder] Dual-Agent GATv2 Actor-Critic architecture. Both agents share a backbone design; action selection is governed by comparing Critic values.

2. **Top-node identification.** Select the top- k nodes most preferred by each agent (by logit magnitude).
3. **Community expansion.** Retrieve the full communities of each selected top node.
4. **Subgraph construction.** Build a reduced subgraph containing only those communities and the target node.
5. **Local operation.** Dynamic feature updates and all subsequent actions operate entirely within this subgraph.

This heuristic trades a small, controllable reduction in F1 score for a dramatic decrease in per-step computation, making previously intractable large graphs feasible. We compare it against a simpler k -neighborhood baseline (extracting the k -hop ego-graph of u), which misses distant “bridge” nodes and performs worse in practice.

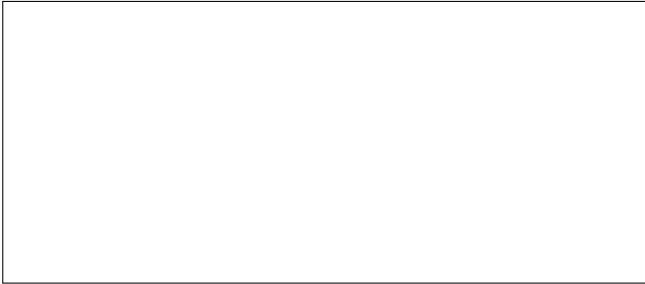


Figure 3: [Placeholder] Policy-Guided Subgraphing: full graph (left) reduced to a task-relevant subgraph (right) guided by agent logits.

4 Overlapping Communities

[This section is left intentionally blank — to be filled in.]

5 Results

We report results using the F1 score (harmonic mean of SR and NMI), following the standard CMH evaluation protocol (bernini-2024). Experiments were run on an AWS EC2 g4dn.xlarge instance (4 Intel Xeon vCPUs, 16 GiB RAM, NVIDIA T4 GPU) over five benchmark datasets of varying

size and density. We swept $\tau \in \{0.3, 0.5, 0.8\}$ and $\beta \in \{0.5, 1.0, 2.0\}$.

5.1 GATv2 Actor-Critic v1 vs. Baselines

Table 1 compares success rates of GATv2 v1 against the original agent across all (τ, β) combinations. Our architecture maintains a consistent and substantial lead across all configurations and does not overfit to any single scenario.

β	τ	GATv2	Original
0.5	0.3	61.67	41.67
	0.5	70.21	38.22
	0.8	79.38	62.67
1.0	0.3	79.17	61.67
	0.5	91.78	58.44
	0.8	96.25	90.00
2.0	0.3	82.08	77.67
	0.5	98.75	74.67
	0.8	99.17	96.33

Table 1: Success rates (%) for GATv2 v1 vs. original agent ($\tau = 0.5$, $\beta = 1$ shown in Figure 4).

Figure 4 shows the F1/time tradeoff for $\tau = 0.5$, $\beta = 1$ across all datasets. GATv2 v1 consistently outperforms all baselines in F1 score, achieving up to 40% improvement over ∇ -CMH on the largest graph. The gradient-based methods are faster but sacrifice F1.

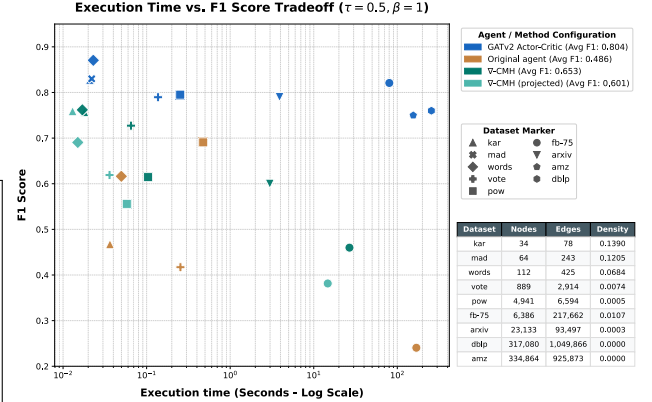


Figure 4: F1 score vs. execution time ($\tau = 0.5$, $\beta = 1$). Time axis in logarithmic scale.

5.2 Dual-Agent Results

[Results for the Dual-Agent (GATv2 v2) to be filled in.]

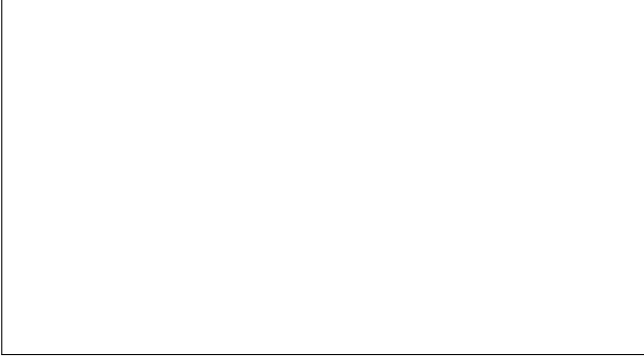


Figure 5: *[Placeholder]* Dual-Agent F1 results across datasets and community detection algorithms ($\tau = 0.5$, $\beta = 1$).



Figure 6: *[Placeholder]* Effect of Policy-Guided Subgraphing on execution time and F1 score on large graphs.